1. FOPL Implementation
2. Water jug problem by production rules
3. Water jug problem by BFS
4. Water jug problem by DFS
5. BFS
6. DFS
7. Puzzle solution
8. Missionaries – cannibals problem by production rules
9. Missionaries – cannibals problem by BFS
10. Missionaries – cannibals problem by DFS
11. Tic – tac – toe problem
12. Best – fit – search
13. A * search
14. Iterative Deepening Depth First Search
15. Maze

1 . FOPL Implementation

```
# Here we import everything from logic.py

from logic import *

rain = Symbol("rain")     # Rain is the symbol for rain
virat = Symbol("virat")
rohit = Symbol("rohit")

# we create here a logical sentence using logical connectives
logical_sentence = And(rain, virat)

# we can't directly print the logical sentence so we use a formula function
print(logical_sentence.formula())

# we create a implication logic here
implication_logic = Implication(Not(rain), virat)

print("implication logic  ", implication_logic)
print(implication_logic.formula())
input("Press a key ....")
```

```
'''We make decision from knowledge base,
all of the statement will be true then
knowlegde base will be true'''

knowledge_base = And(
  Implication(Not(rain), virat),
  Or(virat, rohit),
  Not(And(virat, rohit)),
  virat
)

print(knowledge_base.formula())

''' Here we use the model_check function and here two
arguments are knoledge_base and another
is it rain today or not. Then we print it'''

print(model_check(knowledge_base, rain))
```

logic.py file

```
import itertools
class Sentence():
   def evaluate(self, model):
      """Evaluates the logical sentence."""
      raise Exception("nothing to evaluate")
   def formula(self):
      """Returns string formula representing logical sentence."""
```

```
        return ""
    def symbols(self):
        """Returns a set of all symbols in the logical sentence."""
        return set()
    @classmethod
    def validate(cls, sentence):
        if not isinstance(sentence, Sentence):
            raise TypeError("must be a logical sentence")
    @classmethod
    def parenthesize(cls, s):
        """Parenthesizes an expression if not already parenthesized."""
        def balanced(s):
            """Checks if a string has balanced parentheses."""
            count = 0
            for c in s:
                if c == "(":
                    count += 1
                elif c == ")":
                    if count <= 0:
                        return False
                    count -= 1
            return count == 0
        if not len(s) or s.isalpha() or (
            s[0] == "(" and s[-1] == ")" and balanced(s[1:-1])
        ):
            return s
        else:
            return f"({s})"


class Symbol(Sentence):
    def __init__(self, name):
```

```
        self.name = name
    def __eq__(self, other):
        return isinstance(other, Symbol) and self.name == other.name
    def __hash__(self):
        return hash(("symbol", self.name))

    def __repr__(self):
        return self.name
    def evaluate(self, model):
        try:
            return bool(model[self.name])
        except KeyError:
            raise Exception(f"variable {self.name} not in model")

    def formula(self):
        return self.name
    def symbols(self):
        return {self.name}

class Not(Sentence):
    def __init__(self, operand):
        Sentence.validate(operand)
        self.operand = operand
    def __eq__(self, other):
        return isinstance(other, Not) and self.operand == other.operand
    def __hash__(self):
        return hash(("not", hash(self.operand)))
    def __repr__(self):
        return f"Not({self.operand})"
    def evaluate(self, model):
        return not self.operand.evaluate(model)
```

```
    def formula(self):
        return "¬" + Sentence.parenthesize(self.operand.formula())
    def symbols(self):
        return self.operand.symbols()
class And(Sentence):
    def __init__(self, *conjuncts):
        for conjunct in conjuncts:
            Sentence.validate(conjunct)
        self.conjuncts = list(conjuncts)

    def __eq__(self, other):
        return isinstance(other, And) and self.conjuncts == other.conjuncts



    def __hash__(self):
        return hash(
            ("and", tuple(hash(conjunct) for conjunct in self.conjuncts))
        )
    def __repr__(self):
        conjunctions = ", ".join(
            [str(conjunct) for conjunct in self.conjuncts]
        )
        return f"And({conjunctions})"
    def add(self, conjunct):
        Sentence.validate(conjunct)
        self.conjuncts.append(conjunct)
    def evaluate(self, model):
        return all(conjunct.evaluate(model) for conjunct in self.conjuncts)
    def formula(self):
        if len(self.conjuncts) == 1:
            return self.conjuncts[0].formula()
```

```
        return " ∧ ".join([Sentence.parenthesize(conjunct.formula())
                           for conjunct in self.conjuncts])
    def symbols(self):
        return set.union(*[conjunct.symbols() for conjunct in self.conjuncts])
class Or(Sentence):
    def __init__(self, *disjuncts):
        for disjunct in disjuncts:
            Sentence.validate(disjunct)
        self.disjuncts = list(disjuncts)
    def __eq__(self, other):
        return isinstance(other, Or) and self.disjuncts == other.disjuncts




    def __hash__(self):
        return hash(
            ("or", tuple(hash(disjunct) for disjunct in self.disjuncts))
        )
    def __repr__(self):
        disjuncts = ", ".join([str(disjunct) for disjunct in self.disjuncts])
        return f"Or({disjuncts})"
    def evaluate(self, model):
        return any(disjunct.evaluate(model) for disjunct in self.disjuncts)
    def formula(self):
        if len(self.disjuncts) == 1:
            return self.disjuncts[0].formula()
        return " ∨ ".join([Sentence.parenthesize(disjunct.formula())
                           for disjunct in self.disjuncts])
    def symbols(self):
        return set.union(*[disjunct.symbols() for disjunct in self.disjuncts])
```

```
class Implication(Sentence):
    def __init__(self, antecedent, consequent):
        Sentence.validate(antecedent)
        Sentence.validate(consequent)
        self.antecedent = antecedent
        self.consequent = consequent
        print("Antecendent  : ",antecedent, "consequent  : ",consequent)
        print("In implication class ")
        input("Press any key ....")




    def __eq__(self, other):
        #print("in __eq__")
        return (isinstance(other, Implication)
                and self.antecedent == other.antecedent
                and self.consequent == other.consequent)

    def __hash__(self):
        #print("in __hash__")
        return hash(("implies", hash(self.antecedent), hash(self.consequent)))

    def __repr__(self):
        #print("in __repr__")
        return f"Implication({self.antecedent}, {self.consequent})"

    def evaluate(self, model):
        #print("in evaluate")
        return ((not self.antecedent.evaluate(model))
```

```
            or self.consequent.evaluate(model))

    def formula(self):
        #print("in formula ")
        antecedent = Sentence.parenthesize(self.antecedent.formula())
        consequent = Sentence.parenthesize(self.consequent.formula())
        return f"{antecedent} => {consequent}"

    def symbols(self):
        print("in symbols  ")
        return set.union(self.antecedent.symbols(), self.consequent.symbols())




class Biconditional(Sentence):
    def __init__(self, left, right):
        Sentence.validate(left)
        Sentence.validate(right)
        self.left = left
        self.right = right

    def __eq__(self, other):
        return (isinstance(other, Biconditional)
                and self.left == other.left
                and self.right == other.right)

    def __hash__(self):
        return hash(("biconditional", hash(self.left), hash(self.right)))

    def __repr__(self):
        return f"Biconditional({self.left}, {self.right})"
```

```
    def evaluate(self, model):
        return ((self.left.evaluate(model)
                 and self.right.evaluate(model))
                or (not self.left.evaluate(model)
                    and not self.right.evaluate(model)))

    def formula(self):
        left = Sentence.parenthesize(str(self.left))
        right = Sentence.parenthesize(str(self.right))
        return f"{left} <=> {right}"




    def symbols(self):
        return set.union(self.left.symbols(), self.right.symbols())


def model_check(knowledge, query):
    """Checks if knowledge base entails query."""

    def check_all(knowledge, query, symbols, model):
        """Checks if knowledge base entails query, given a particular model."""

        print("in check all..")
        input("check all exit...")
        # If model has an assignment for each symbol
        if not symbols:

            # If knowledge base is true in model, then query must also be true
            if knowledge.evaluate(model):
                return query.evaluate(model)
```

```
        return True
    else:
        # Choose one of the remaining unused symbols
        remaining = symbols.copy()
        p = remaining.pop()
        # Create a model where the symbol is true
        model_true = model.copy()
        model_true[p] = True
        # Create a model where the symbol is false
        model_false = model.copy()
        model_false[p] = False
        # Ensure entailment holds in both models
        return (check_all(knowledge, query, remaining, model_true) and
                check_all(knowledge, query, remaining, model_false))

# Get all symbols in both knowledge and query
symbols = set.union(knowledge.symbols(), query.symbols())

# Check that knowledge entails query
return check_all(knowledge, query, symbols, dict())
```

2. Water jug problem by production rules

```
from collections import deque
def Solution(a, b, target):
    m = {}
    isSolvable = False
    path = []
    q = deque()
    #Initializing with jugs being empty
    q.append((0, 0))
    while (len(q) > 0):
        # Current state
        u = q.popleft()
        if ((u[0], u[1]) in m):
            continue
        if ((u[0] > a or u[1] > b or
            u[0] < 0 or u[1] < 0)):
            continue
        path.append([u[0], u[1]])
        m[(u[0], u[1])] = 1
        if (u[0] == target or u[1] == target):
            isSolvable = True
            if (u[0] == target):
                if (u[1] != 0):
                    path.append([u[0], 0])
            else:
                if (u[0] != 0):
                    path.append([0, u[1]])
```

```
            sz = len(path)
            for i in range(sz):
                print("(", path[i][0], ",", path[i][1], ")")
            break
        q.append([u[0], b]) # Fill Jug2
        q.append([a, u[1]]) # Fill Jug1
        for ap in range(max(a, b) + 1):
            c = u[0] + ap
            d = u[1] - ap
            if (c == a or (d == 0 and d >= 0)):
                q.append([c, d])
            c = u[0] - ap
            d = u[1] + ap
            if ((c == 0 and c >= 0) or d == b):
                q.append([c, d])
        q.append([a, 0])
        q.append([0, b])
    if (not isSolvable):
        print("Solution not possible")
Jug1, Jug2, target = 4, 3, 2
print("Path from initial state to solution state :")
Solution(Jug1, Jug2, target)
```

3. Water jug problem by BFS

```
from collections import deque


def bfs_water_jug(capacity, target):
    # Initialize the queue and visited set
    initial_state = (0, 0)
    queue = deque([(initial_state, [initial_state])])  # Store state and path
    visited = set()
    visited.add(initial_state)
    while queue:
        (jug1, jug2), path = queue.popleft()
        # Check if we have reached the target (0, target)
        if jug1 == 0 and jug2 == target:
            return path  # Return the path to the solution
        # List of all possible next states
        possible_states = [
            (capacity[0], jug2),  # Fill jug1
            (jug1, capacity[1]),  # Fill jug2
            (0, jug2),          # Empty jug1
            (jug1, 0),          # Empty jug2
            (min(jug1 + jug2, capacity[0]), jug1 + jug2 - min(jug1 + jug2, capacity[0])),  # Pour jug2 into jug1
            (jug1 + jug2 - min(jug1 + jug2, capacity[1]), min(jug1 + jug2, capacity[1]))  # Pour jug1 into jug2
        ]
        for state in possible_states:
            if state not in visited:
                visited.add(state)
                queue.append((state, path + [state]))  # Append new state and the path
    return None  # No solution found
```

```
# Define jug capacities and target amount
capacity = (4, 3)  # Capacities of jug1 and jug2
target = 2         # Target amount of water in jug2
# Find the path to measure the target amount
solution_path = bfs_water_jug(capacity, target)
if solution_path:
    print("Path to measure (0,", target, ") liters:")
    for step in solution_path:
        print(step)
else:
    print("No solution found to measure (0,", target, ") liters.")
```

4. Water jug problem by DFS

```
def dfs_water_jug(capacity, target, state, path, visited):
    jug1, jug2 = state
    # Check if we have reached the target (0, target)
    if jug1 == 0 and jug2 == target:
        return path + [state]
    visited.add(state)
    # List of all possible next states
    possible_states = [
        (capacity[0], jug2),  # Fill jug1
        (jug1, capacity[1]),  # Fill jug2
        (0, jug2),            # Empty jug1
        (jug1, 0),            # Empty jug2
        (min(jug1 + jug2, capacity[0]), jug1 + jug2 - min(jug1 + jug2, capacity[0])),  # Pour jug2 into jug1
        (jug1 + jug2 - min(jug1 + jug2, capacity[1]), min(jug1 + jug2, capacity[1]))  # Pour jug1 into jug2
    ]
    for next_state in possible_states:
        if next_state not in visited:
            result = dfs_water_jug(capacity, target, next_state, path + [state], visited)
            if result:  # If a valid path is found, return it
                return result
    return None  # No solution found
# Define jug capacities and target amount
capacity = (4, 3)  # Capacities of jug1 and jug2
target = 2         # Target amount of water in jug2
initial_state = (0, 0)  # Start with both jugs empty
# Use DFS to find the path to measure the target amount
visited = set()
solution_path = dfs_water_jug(capacity, target, initial_state, [], visited)
```

```
if solution_path:
    print("Path to measure (0,", target, ") liters:")
    for step in solution_path:
        print(step)
else:
    print("No solution found to measure (0,", target, ") liters.")
```

5.BFS

```
# Breath First search program
graph = {
   'A' : ['B' , 'C'],
   'B' : ['D' , 'E'],
   'C' : ['F' , 'G'],
   'D' : [],
   'E' : [],
   'F' : [] }
visited = []
queue = []
goal = 'F'
def BFS(visited , graph , node):
   visited.append(node)
   queue.append(node)
   while queue:
      S = queue.pop(0)
      print("S : " ,  S)
      input("PRESS A KEY ....")
      for neighbour in graph[S]:
         if neighbour not in visited:
            print("VISITED : ",visited)
            print("NEIGHBOUR : ",neighbour)
            print("QUEUE : " ,queue)
            input("PRESS A KEY .....")
            visited.append(neighbour)
            queue.append(neighbour)
         if goal in visited:
            break
BFS(visited , graph , 'A')
```

6.DFS

```
# Depth First search program
graph={
   'A':['B','C'],
   'B':['D','E'],
   'C':['F','G'],
   'D':[],
   'E':[],
   'F':[]  }
visited=[]
queue=[]
goal='F'
def DFS (visited,graph,node):
   visited.append(node)
   queue.append(node)
   while queue:
      S=queue.pop(-1)
      print("S:",S)
      input("press a key...")
      for neighbour in graph[S]:
         if neighbour not in visited:
            print("VISITED:",visited)
            print("NEIGHBOUR:",neighbour)
            print("QUEUE:",queue)
            input("press a key....")
            visited.append(neighbour)
            queue.append(neighbour)
         if goal in visited:
            break
DFS(visited,graph,'A')
```

7. Puzzle solution

```
from collections import deque
class PuzzleState:
    def __init__(self, board, zero_pos, moves=0, previous=None):
        self.board = board
        self.zero_pos = zero_pos
        self.moves = moves
        self.previous = previous
    def get_neighbors(self):
        neighbors = []
        x, y = self.zero_pos
        directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]
        for dx, dy in directions:
            nx, ny = x + dx, y + dy
            if 0 <= nx < 3 and 0 <= ny < 3:
                new_board = [row[:] for row in self.board]
                new_board[x][y], new_board[nx][ny] = new_board[nx][ny], new_board[x][y]
                neighbors.append(PuzzleState(new_board, (nx, ny), self.moves + 1, self))
        return neighbors
    def __hash__(self):
        return hash(tuple(map(tuple, self.board)))
    def __eq__(self, other):
        return self.board == other.board
    def print_board(self):
        for row in self.board:
            print(' '.join(str(x) if x != 0 else ' ' for x in row))
        print()
def bfs_solver(initial_board):
    zero_pos = next((i, j) for i in range(3) for j in range(3) if initial_board[i][j] == 0)
    start_state = PuzzleState(initial_board, zero_pos)
```

```
    queue = deque([start_state])
    visited = {start_state}
    solution_path = []
    while queue:
        current_state = queue.popleft()
        # Check if we reached the goal state
        if current_state.board == [[1, 2, 3], [4, 5, 6], [7, 8, 0]]:
            solution_path.append(current_state)
            while current_state:
                solution_path.append(current_state)
                current_state = current_state.previous
            return solution_path[::-1]  # Return reversed path
        for neighbor in current_state.get_neighbors():
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return []  # No solution
# Example usage
initial_board = [
    [1, 2, 3],
    [4, 0, 5],
    [7, 8, 6]
]
solution = bfs_solver(initial_board)
if solution:
    print("Minimum moves to solve the puzzle:", len(solution) - 1)
    for state in solution:
        state.print_board()
else:
    print("No solution found.")
```

8.Missionaries-cannibal problem by production rules

```
class State:
    def __init__(self, m_left, c_left, m_right, c_right, boat):
        self.m_left = m_left  # Number of missionaries on the left bank
        self.c_left = c_left  # Number of cannibals on the left bank
        self.m_right = m_right  # Number of missionaries on the right bank
        self.c_right = c_right  # Number of cannibals on the right bank
        self.boat = boat  # Position of the boat (1 for left, 0 for right)
    def is_valid(self):
        # Valid state if there are no more cannibals than missionaries on either side
        if self.m_left < 0 or self.c_left < 0 or self.m_right < 0 or self.c_right < 0:
            return False
        if (self.m_left > 0 and self.m_left < self.c_left) or (self.m_right > 0 and self.m_right < self.c_right):
            return False
        return True
    def __str__(self):
        return f"Left: M={self.m_left}, C={self.c_left} | Right: M={self.m_right}, C={self.c_right} | Boat: {'Left' if self.boat else 'Right'}"
    def __eq__(self, other):
        return (self.m_left, self.c_left, self.m_right, self.c_right, self.boat) == (other.m_left, other.c_left, other.m_right, other.c_right, other.boat)
    def __hash__(self):
        return hash((self.m_left, self.c_left, self.m_right, self.c_right, self.boat))
def production_rules(state):
    actions = []
    if state.boat == 1:  # Boat on the left bank
        # Possible actions (moving to the right)
        if state.m_left > 0:  # Move 1 missionary
            actions.append(State(state.m_left - 1, state.c_left, state.m_right + 1, state.c_right, 0))
        if state.m_left > 1:  # Move 2 missionaries
```

```
            actions.append(State(state.m_left - 2, state.c_left, state.m_right + 2, state.c_right, 0))
        if state.c_left > 0:  # Move 1 cannibal
            actions.append(State(state.m_left, state.c_left - 1, state.m_right, state.c_right + 1, 0))
        if state.c_left > 1:  # Move 2 cannibals
            actions.append(State(state.m_left, state.c_left - 2, state.m_right, state.c_right + 2, 0))
        if state.m_left > 0 and state.c_left > 0:  # Move 1 missionary and 1 cannibal
            actions.append(State(state.m_left - 1, state.c_left - 1, state.m_right + 1, state.c_right + 1,
0))
    else:  # Boat on the right bank
        # Possible actions (moving to the left)
        if state.m_right > 0:  # Move 1 missionary
            actions.append(State(state.m_left + 1, state.c_left, state.m_right - 1, state.c_right, 1))
        if state.m_right > 1:  # Move 2 missionaries
            actions.append(State(state.m_left + 2, state.c_left, state.m_right - 2, state.c_right, 1))
        if state.c_right > 0:  # Move 1 cannibal
            actions.append(State(state.m_left, state.c_left + 1, state.m_right, state.c_right - 1, 1))
        if state.c_right > 1:  # Move 2 cannibals
            actions.append(State(state.m_left, state.c_left + 2, state.m_right, state.c_right - 2, 1))
        if state.m_right > 0 and state.c_right > 0:  # Move 1 missionary and 1 cannibal
            actions.append(State(state.m_left + 1, state.c_left + 1, state.m_right - 1, state.c_right - 1,
1))
    return [action for action in actions if action.is_valid()]
def solve_missionaries_and_cannibals():
    initial_state = State(3, 3, 0, 0, 1)  # Initial state: 3 M, 3 C on the left, boat on left
    goal_state = State(0, 0, 3, 3, 0)  # Goal state: 0 M, 0 C on the left, boat on right
    states = [initial_state]  # Start with the initial state
    visited = set()  # Keep track of visited states
    path = []
    while states:
        current_state = states.pop(0)
        path.append(current_state)
```

```
        if current_state == goal_state:
            return path  # Goal reached

        visited.add(current_state)

        for next_state in production_rules(current_state):
            if next_state not in visited:  # Avoid cycles
                states.append(next_state)

    return None  # No solution found

# Example usage
result = solve_missionaries_and_cannibals()

# Display the steps to reach the goal state
if result:
    print("Steps to solve the Missionaries and Cannibals problem:")
    for step in result:
        print(step)
else:
    print("No solution found.")
```

9.Missionaries-cannibal problem by BFS

```
def is_valid(state):
    ml, cl, boat_pos, mr, cr = state
    if ml >= 0 and cl >= 0 and mr >= 0 and cr >= 0:
        if (ml == 0 or ml >= cl) and (mr == 0 or mr >= cr):
            return True
        return False
def bfs():
    start = (3, 3, 0, 0, 0)
    goal = (0, 0, 1, 3, 3)
    queue = [(start, [])]
    visited = set([start])
    moves = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]
    while queue:
        curr_state, path = queue.pop(0) # dic
        if curr_state == goal:
            return path + [curr_state]
        ml, cl, boat_pos, mr, cr = curr_state
        for m_move, c_move in moves:
            if boat_pos == 0: # left side
                new_state = (ml - m_move, cl - c_move, 1, mr + m_move, cr + c_move)
            else: # right side
                new_state = (ml + m_move, cl + c_move, 0, mr - m_move, cr - c_move)
            if is_valid(new_state) and new_state not in visited:
                visited.add(new_state)
                queue.append((new_state, path + [curr_state]))
    return None
#run
sol = bfs()
if sol:  for I in sol: print(i) else:print(“no”)
```

10.Missionaries-cannibal problem by DFS

```
def is_valid(state):
    ml, cl, boat_pos, mr, cr = state
    if ml >= 0 and cl >= 0 and mr >= 0 and cr >= 0:
        if (ml == 0 or ml >= cl) and (mr == 0 or mr >= cr):
            return True
        return False
def bfs():
    start = (3, 3, 0, 0, 0)
    goal = (0, 0, 1, 3, 3)
    queue = [(start, [])]
    visited = set([start])
    moves = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]
    while queue:
        curr_state, path = queue.pop(-1) # dic
        if curr_state == goal:
            return path + [curr_state]
        ml, cl, boat_pos, mr, cr = curr_state
        for m_move, c_move in moves:
            if boat_pos == 0: # left side
                new_state = (ml - m_move, cl - c_move, 1, mr + m_move, cr + c_move)
            else: # right side
                new_state = (ml + m_move, cl + c_move, 0, mr - m_move, cr - c_move)
            if is_valid(new_state) and new_state not in visited:
                visited.add(new_state)
                queue.append((new_state, path + [curr_state]))
    return None
#run
sol = bfs()
if sol:  for I in sol: print(i) else:(“no”)
```

11. Tic-Tac-Toe

```
b=[" "]*9
def show_b():
    print("---------")
    for i in range(3):
        print("| " + " | ".join(b[i*3:(i+1)*3]) + " |")
        print("---------")
def win(p):
    w=[(0,1,2),(3,4,5),(6,7,8),(0,3,6),(1,4,7),(2,5,8),(0,4,8),(2,4,6)]
    return any(all(b[i]==p for i in t) for t in w)
def full():
    return " " not in b
def player_turn():
    while True:
        try:
            m=int(input("Your move (1-9): "))-1
            if b[m]==" ":
                b[m]="X"
                break
            else:
                print("Spot taken!")
        except:
            print("Invalid input!")
def comp_turn():
    for i in range(9):
        if b[i]==" ":
            b[i]="O"
            break
def play():
    print("Tic-Tac-Toe! You: X, Computer: O")
```

```
    show_b()
    while True:
        player_turn()
        show_b()
        if win("X"):
            print("You win!")
            break
        if full():
            print("It's a draw!")
            break
        comp_turn()
        print("Computer's move:")
        show_b()
        if win("O"):
            print("Computer wins!")
            break
        if full():
            print("It's a draw!")
            break
play()
```

12.Best- fit-search

```
from queue import  PriorityQueue
v = 14
graph = [[] for i in range(v)]
def addedge(x,y,cost):
    graph[x].append((y,cost))
    graph[y].append((x,cost))
addedge(0,1,3)
addedge(0,2,6)
addedge(0,3,5)
addedge(1,4,9)
addedge(1,5,8)
addedge(2,6,12)
addedge(2,7,14)
addedge(3,8,7)
addedge(8,9,5)
addedge(8,10,6)
addedge(9,11,1)
addedge(9,12,10)
addedge(9,13,2)
def best_fit_search(actual_src, target , n):
    visited = [False] * n
    pq = PriorityQueue()
    pq.put((0, actual_src))
    visited[actual_src] = True
    while pq.empty() == False:
        u = pq.get()[1]
        print(u,end=" ")
        if u == target:
            print("\nTarget reached : ",u)
```

```
            break
        for v,c in graph[u]:
            if visited[v]==False:
                visited[v] == True
                pq.put((c,v))
                print("\nVisited : ", v)
                print("Cost : ",c)

source = 0
target = 9
best_fit_search(source,target,v)
```

13. A* search

```
graph_nodes={
   'A':[('B',6),('F',3)],
   'B':[('C',3),('D',2)],
   'C':[('D',1),('E',5)],
   'D':[('C',1),('E',8)],
   'E':[('I',5),('J',5)],
   'F':[('G',1),('H',7)],
   'G':[('I',3)],
   'H':[('I',2)],
   'I':[('E',5),('J',3)]
   }
def get_neighbours(v):
   if v in graph_nodes:
      return graph_nodes[v]
   else:
      return None
def h(n):
   H_list={
      'A':10,
      'B':8,
      'C':5,
      'D':7,
      'E':3,
      'F':6,
      'G':5,
      'H':3,
      'I':1,
      'J':0
      }
```

```
    return H_list[n]
def astarAlgo(start_node,stop_node):
    open_set=set(start_node)
    closed_set=set()
    g={}
    parents={}
    g[start_node]=0
    parents[start_node]=start_node
    while len(open_set)>0:
        n=None
        for v in open_set:
            if n==None or g[v]+h(v)<g[n]+h(n):
                n=v
        if n==stop_node or graph_nodes[n]==None:
            pass
        else:
            for (m,weight) in get_neighbours(n):
                if m not in open_set and m not in closed_set:
                    open_set.add(m)
                    parents[m]=n
                    g[m]=g[n]+weight
                else:
                    if g[m]>g[n]+weight:
                        g[m]=g[n]+weight
                        parents[m]=n
                        if m in closed_set:
                            closed_set.remove(m)
                            open_set.add(m)
        if n==None:
            print("Path does not exists....")
            return None
```

```
        if n==stop_node:
            path=[]
            while parents[n]!=n:
                path.append(n)
                n=parents[n]
            path.append(start_node)
            path.reverse()
            print("path found..{}".format(path))
            return path
        open_set.remove(n)
        closed_set.add(n)
    print("Path does not exists...")
    return None

astarAlgo('A','J')
```

14.IDDFS

```
class Graph:
    def __init__(self):
        self.graph = {}
    def add_edge(self, u, v):
        if u not in self.graph:
            self.graph[u] = []
        self.graph[u].append(v)
    def depth_limited_dfs(self, node, goal, depth):
        if depth == 0 and node == goal:
            return True
        if depth > 0:
            for neighbor in self.graph.get(node, []):
                if self.depth_limited_dfs(neighbor, goal, depth - 1):
                    return True
        return False
    def iterative_deepening_dfs(self, start, goal):
        depth = 0
        while True:
            print(f"Searching at depth: {depth}")
            if self.depth_limited_dfs(start, goal, depth):
                return True
            depth += 1
# Example usage
g = Graph()
g.add_edge(0, 1)
g.add_edge(0, 2)
g.add_edge(1, 3)
g.add_edge(1, 4)
g.add_edge(2, 5)
```

```
g.add_edge(2, 6)
start_node = 0
goal_node = 4
if g.iterative_deepening_dfs(start_node, goal_node):
    print(f"Goal {goal_node} found!")
else:
    print(f"Goal {goal_node} not found.")
```

15. Maze

```
EE = "XX"
EY = "--"
# Only One Condition Is Maze Is In Square Form
maze = [
   [EE, EE, EE, EE, EY],
   [EE, EE, EE, EE, EY],
   [EY, EY, EE, EE, EY],
   [EE, EY, EE, EY, EY],
   [EE, EY, EY, EY, EE],
]
for i in maze:
   for j in i :
      print(j , end= " ")
   print()
s1, s2 = 2, 0
e1, e2 = 0, 4
mazeSize = len(maze)
visited = []
queue = []
visited.append([s1, s2])
queue.append([s1 , s2])
def findNebiourNode(arr):
   c, d = arr[0], arr[1]
   neb = []
   if c - 1 < mazeSize and d < mazeSize:
      neb.append([c - 1, d])
   if c < mazeSize and d + 1 < mazeSize:
      neb.append([c , d + 1])
   if c + 1 < mazeSize and d < mazeSize:
```

```
        neb.append([c + 1, d])
    if c < mazeSize and d - 1 >= 0:
        neb.append([c , d - 1])
    return neb
while queue:
    childNeb = findNebiourNode(queue.pop(0))
    for neb in childNeb:
        if neb[0] == e1 and neb[1] == e2:
            visited.append([neb[0], neb[1]])
            print(visited)
            print("Congratulation You Win!")
            break
        elif maze[neb[0]][neb[1]] == EY and neb not in visited:
            queue.append([neb[0], neb[1]])
            visited.append([neb[0], neb[1]])
```